

Foundations of Cybersecurity

5 July 2022

Exercise n. 1 [10]

Explain the RSA key generation algorithm, discussing the complexity of each step.

Exercise n. 2 [12]

Alice wishes to protect passwords from an attacker who manages to grab the password file and then perform an off-line attack, possibly based on a rainbow table attack. For this reason, Alice decides to salt the hashing. Let p denote a plaintext password and s a 128-bit random salt. Alice adopts the following salting scheme.

Scheme 1: Compute $h = \text{MD5}^{1000}(p) \oplus s$, and store the pair (h, s) in the password file.

A. Discuss the security of this scheme w.r.t. a rainbow table attack.

Bob suggests Alice to employ the following salting scheme:

Scheme 2: Compute $h = \text{MD5}^{1000}(p \parallel s)$, and store (h, s) in the password file.

B. Is this choice better or worse than Alice's w.r.t. a rainbow table attack?

Within Scheme 2, assume that users employ 8 characters chosen over the English lowercase alphanumeric characters.

C. How many bits long should be the salt to prevent an attacker able to compute 2^{70} passwords from employing a rainbow table attack?

D. Does the previous amount of random salt prevent an attacker from brute forcing a single password?

Exercise n. 3 [8]

Find and explain the vulnerabilities of the following function. Then patch them.

```
void do_stuff(int aux) {
    if (aux < 0) {
        /*Handle error*/
    }
    unsigned char* buf = (char*) malloc(aux);
    if (gets(buf) == NULL) {
        /* Handle error */
    }
    /* do other stuff */
}
```

Solution

Exercise n.1

See theory.

Exercise n.2

- A. The chosen salted hash strategy allows an attacker to easily remove the effect of the salt through simply computing $\lambda = h \oplus s$ and then search λ in the rainbow table.
- B. The proposed scheme is a better one: there is no trivial way to remove the salt from the hash, so the hacker must try all the possible pairs (password, salt). A rainbow table for each value of salt s is not viable.
- A. Considering that an 8 characters password over lowercase alphanumeric characters lies in a $(26 + 10)^8 \approx 2^{41}$ wide password space, we need to employ a salt longer than $70 - 41 = 29$ bits to prevent an attacker to mount a rainbow table attack precomputing all the possible hashes for all the possible salts.
- B. No: the cost for brute forcing a single password is 2^{41} operations that is nowadays feasible.

Exercise n.3

In this exercise there are two issues.

- 1. The code does not check if the malloc fails;¹
- 2. If `aux == 0`, the behaviour of `malloc` is allocation dependent: it may return both a null or a non-null pointer. In the latter case the pointer cannot be dereferenced and should be passed to `free` to avoid memory leaks.
- 3. The function `gets()` is deprecated, because the programmer cannot specify the amount of bytes to transfer from the input stream to the buffer.² Therefore, a possible solution is the following

```
void do_stuff(int aux) {
    if (aux <= 0){
        /*Handle Error*/;}
    unsigned char* buf = (char*) malloc(aux);
    if(buf==NULL){/*handle error*/}
    if (fgets(buf, aux, stdin) == NULL) {
        /* Handle error */
        free(buf) ;
        return;}
    char* p = strchr(buf, '\n');
    if (p) {*p = '\0'; }
    /* do other stuff */
}
```

¹ On failure, returns a null pointer (<https://en.cppreference.com/w/c/memory/malloc>).

² The `gets()` function does not perform bounds checking, therefore this function is extremely vulnerable to buffer-overflow attacks. It cannot be used safely (unless the program runs in an environment which restricts what can appear on `stdin`). For this reason, the function has been deprecated in the third corrigendum to the C99 standard and removed altogether in the C11 standard (<https://en.cppreference.com/w/c/io/fgets>).

N.B.: to replace the character `'\n'` with a terminating character, is not strictly necessary, therefore it can be considered *not an error*.